

CC3092 – Deep Learning

Proyecto 2

Instrucciones

- Esta es una actividad en grupos de no más de 3 integrantes.
 - Recuerden **unirse al grupo de canvas**
- No se permitirá ni se aceptará cualquier indicio de copia. De presentarse, se procederá según el reglamento correspondiente.
- Tendrán hasta el día indicado en Canvas.
 - No se confíen, aprovechen el tiempo en clase para entender todos los ejercicios y avanzar lo más posible.
- **NOTA:** Limiten el uso de IA generativa. Intenten primero buscar en fuentes de internet y si en verdad necesitan usarla, asegúrense de **colcar el prompt** que utilizan para cada task donde corresponda, así como una explicación de por qué ese prompt funcionó.

Guatemala recibe más de \$20 mil millones anuales en remesas, equivalentes al 19% del PIB nacional. Ese volumen convierte al corredor de remesas Guatemala-Estados Unidos en uno de los más importantes del mundo y, simultáneamente, en uno de los más vulnerables a ser utilizado para lavado de dinero y financiamiento de actividades ilícitas. Las empresas que procesan remesas (Western Union, MoneyGram, Remitly) y los bancos que las reciben tienen obligaciones regulatorias de detectar patrones sospechosos bajo marcos como la Ley Contra el Lavado de Dinero u Otros Activos de Guatemala y las regulaciones FinCEN en Estados Unidos.

El problema técnico central es que los sistemas de detección actuales basados en reglas tienen tasas de falsos positivos del 95-99%, lo que significa que los equipos de cumplimiento deben revisar manualmente cientos de alertas para encontrar un caso real. Eso es insostenible a escala y deja espacio para que patrones sofisticados de lavado pasen desapercibidos.

Este proyecto propone construir un sistema de detección inteligente que aprenda qué es un patrón normal de remesas antes de aprender qué es sospechoso, produciendo alertas con trazabilidad: el sistema debe poder explicar qué transacciones específicas de un remitente activaron la alerta..

Datos

Dataset principal: PaySim — Synthetic Financial Dataset for Fraud Detection, disponible en Kaggle. Generado por el Banco Mundial a partir de datos reales de un servicio de dinero móvil africano, con 6.3 millones de transacciones financieras sintéticas que replican patrones reales de uso y de fraude.

Dataset complementario: IBM AML Anti Money Laundering, disponible en Kaggle. Diseñado específicamente para investigación en detección de lavado de dinero con patrones etiquetados por expertos en cumplimiento.

Acceso:

PaySim

<https://www.kaggle.com/datasets/ealaxi/paysim1>

IBM AML

<https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>

Instrucción importante: los datos son masivos. El grupo debe tomar decisiones explícitas y justificadas sobre qué subconjunto usar, cómo construir las secuencias por remitente, y cómo balancear el dataset para el entrenamiento. Esas decisiones deben documentarse en el reporte con justificación técnica, no simplemente ejecutarse sin explicación

Estructura del Proyecto

El proyecto tiene cuatro componentes con pesos distintos. Los componentes 1 y 2 son técnicos y se entregan como notebook ejecutado. El componente 3 es analítico y se entrega como reporte escrito. El componente 4 es el MVP funcional.

Componente 1: Ingeniería de datos y representación de secuencias

El primer desafío del proyecto no es modelar sino representar. Una transacción aislada contiene poca información sobre la intención del remitente. Lo que revela lavado de dinero es el patrón a lo largo del tiempo: la frecuencia inusual, los montos que se fragmentan justo debajo de umbrales regulatorios, los destinos que cambian abruptamente, la concentración en horarios específicos.

El grupo debe construir un pipeline que transforme las transacciones individuales en secuencias por remitente, donde cada secuencia capture el comportamiento histórico de ese remitente como una serie ordenada temporalmente.

Las decisiones que el grupo debe tomar y documentar:

- Qué features representan cada transacción dentro de la secuencia (monto, tipo, hora, destino, frecuencia, etc.) y cómo se normalizan
- Qué longitud máxima de secuencia usar y qué hacer con remitentes que tienen muy pocas transacciones
- Cómo manejar el desbalance extremo entre remitentes normales y sospechosos
- Cómo separar entrenamiento, validación y prueba sin filtración de datos entre splits

Verificación: el notebook debe mostrar explícitamente la distribución de longitudes de secuencia, la proporción de casos positivos y negativos, y al menos tres ejemplos visualizados de secuencias normales versus sospechosas antes de cualquier modelado.

Componente 2: Sistema de detección en dos etapas

El núcleo técnico del proyecto. El grupo debe diseñar e implementar un sistema que aborde el problema desde dos perspectivas complementarias y combine sus señales en una predicción final.

Etapas A: Aprendizaje de la normalidad

Antes de intentar clasificar qué es sospechoso, el sistema debe aprender qué es normal. La idea es que un modelo entrenado exclusivamente sobre comportamiento normal de remesas aprenderá una representación comprimida de ese comportamiento. Cuando se le presente una secuencia que no se parece a nada que haya visto, fallará en reproducirla correctamente. Esa dificultad de reproducción es la señal de anomalía.

El grupo debe seleccionar e implementar una arquitectura apropiada para este objetivo. La arquitectura debe ser capaz de procesar secuencias de longitud variable, producir una representación comprimida del

comportamiento del remitente, y reconstruir la secuencia original desde esa representación. El error de reconstrucción por secuencia se convierte en el score de anomalía.

Una vez entrenado el modelo, el grupo debe definir un umbral sobre el error de reconstrucción que separe comportamiento normal de anómalo. Esa decisión no es arbitraria: debe justificarse con una métrica de evaluación apropiada sobre el conjunto de validación.

Etapas B: Aprendizaje supervisado sobre casos etiquetados

Con el conocimiento de representación aprendido en la Etapa A, el grupo debe construir un clasificador que aprenda a distinguir lavado de dinero de comportamiento legítimo usando los casos etiquetados disponibles. El clasificador debe aprovechar lo aprendido en la Etapa A en lugar de aprender desde cero, aplicando alguna de las estrategias de transfer learning estudiadas en la Semana 9.

El grupo debe justificar explícitamente qué estrategia de adaptación eligió y por qué, qué función de pérdida es apropiada dado el desbalance de clases y la naturaleza del problema, y cómo combina las señales de ambas etapas en una predicción final.

Experimento de demostración obligatorio:

El grupo debe demostrar empíricamente que la arquitectura de dos etapas aporta valor sobre una línea base de clasificador supervisado entrenado desde cero sin la Etapa A. Sin ese experimento, el proyecto no está técnicamente completo. Los resultados de demostración deben presentarse en una tabla comparativa con la métrica o métricas que el grupo considere más apropiadas para este problema, con justificación de esa elección.

Componente 3: Análisis, interpretabilidad y reporte

Un sistema de detección de lavado de dinero que no puede explicar sus decisiones no puede usarse en un entorno regulatorio real. Los equipos de cumplimiento necesitan saber por qué una alerta fue generada antes de iniciar una investigación. Este componente evalúa la profundidad del análisis y la calidad del reporte.

Análisis de interpretabilidad

El grupo debe analizar qué transacciones específicas de una secuencia activaron la alerta del sistema. Esto implica examinar los pesos de atención del modelo sobre la secuencia y conectar los patrones encontrados con tipologías conocidas de lavado de dinero en remesas. El grupo debe seleccionar al menos cinco casos del conjunto de prueba, tres correctamente detectados y dos falsos positivos o falsos negativos, y analizar en profundidad qué hizo el modelo en cada caso y por qué.

Reporte ejecutivo

El reporte debe tener entre 2,000 y 3,000 palabras y cubrir:

- El problema de negocio en contexto regulatorio guatemalteco y centroamericano, con referencias a la legislación AML vigente
- Las decisiones de diseño del sistema con justificación técnica y de negocio
- Los resultados del experimento de ablación con análisis de qué aporta cada etapa
- El análisis de casos específicos detectados y no detectados

- Las limitaciones del sistema y qué se necesitaría para llevarlo a producción en una empresa de remesas real
- Al menos tres referencias a papers de venues indexadas (NeurIPS, ICML, ICLR, ACL, o journals de compliance financiero) publicados entre 2020 y 2025

El reporte debe poder ser leído por un director de cumplimiento de un banco guatemalteco que no es experto en deep learning. Eso significa que los conceptos técnicos deben explicarse con lenguaje de negocio y los resultados deben traducirse a implicaciones operativas concretas.

Componente 4: MVP funcional

El grupo debe construir una interfaz funcional que demuestre el sistema en uso. La interfaz puede ser un artifact de Claude, una aplicación Streamlit, o cualquier herramienta que produzca una experiencia interactiva sin requerir instalación del evaluador.

La interfaz debe permitir:

- Ingresar o seleccionar un remitente del conjunto de prueba
- Visualizar su secuencia de transacciones con información básica de cada una
- Ver el score de anomalía de la Etapa A y la probabilidad de lavado de la Etapa B
- Ver un mapa de calor sobre la secuencia que muestre qué transacciones contribuyeron más a la alerta
- Generar automáticamente un párrafo de explicación en lenguaje natural del por qué de la alerta

El MVP no necesita ser visualmente elaborado pero debe funcionar sin errores en el momento de la presentación. Un MVP que no corre vale cero puntos independientemente de la calidad del resto del proyecto.

Entregables

Tres archivos subidos a la plataforma antes del domingo 20 de septiembre a las 11:59 PM:

1. Notebook ejecutado (.ipynb): con todas las celdas con salida visible, código comentado, y los experimentos de ablación claramente identificados. El notebook debe poder reproducirse desde cero en menos de 30 minutos en Google Colab con GPU T4 gratuita.

2. Reporte (.pdf): el documento ejecutivo descrito en el Componente 3, con las referencias en formato APA o IEEE.

3. Enlace al MVP: URL de la interfaz funcional en un archivo de texto (.txt). Si es un artifact de Claude debe ser público compartido en enlace al mismo. Si es Streamlit debe estar desplegado en Streamlit Cloud (gratuito).

Evaluación

Componente	Criterio	Pts
C1: Ingeniería de datos	Secuencias bien construidas, decisiones documentadas, visualizaciones	20
C2A: Etapa A	Arquitectura apropiada, entrenamiento solo sobre normalidad, umbral justificado	20
C2B: Etapa B	Transfer learning desde Etapa A, pérdida justificada, ablation completo	20
C3: Análisis	Interpretabilidad de atención, análisis de casos, reporte ejecutivo	25
C4: MVP	Funciona sin errores, muestra ambas etapas, mapa de calor, explicación en lenguaje natural	15
Total		100

Criterios de evaluación transversales

Estos criterios aplican a todos los componentes y pueden sumar o restar puntos del total:

Justificación de decisiones: cada decisión de diseño (arquitectura, función de pérdida, umbral, estrategia de transfer learning, métricas) debe tener una justificación explícita. Una decisión sin justificación se evalúa como si fuera incorrecta aunque el resultado sea bueno.

Conexión con el curso: el reporte y el notebook deben conectar explícitamente las técnicas usadas con las semanas del curso donde se desarrollaron. No basta implementar: el grupo debe demostrar que entiende por qué esa técnica es apropiada para este problema en términos de lo que aprendió.

Honestidad sobre limitaciones: un proyecto que reporta honestamente qué no funcionó y por qué es más valioso que uno que solo reporta lo que salió bien. El análisis de fallos es parte de la ingeniería.

Uso de IA generativa: el uso de herramientas como Claude, ChatGPT o Gemini está permitido y es esperado. Sin embargo, el grupo debe incluir al final del reporte una sección de máximo 200 palabras describiendo cómo usaron esas herramientas y qué decisiones tomaron ellos mismos. Un proyecto donde la IA tomó todas las decisiones de diseño sin criterio propio del grupo se penaliza en los criterios de justificación.

Preguntas frecuentes anticipadas

¿Pueden usar librerías de alto nivel como HuggingFace Transformers? Para el componente de NLP pueden usar modelos preentrenados de HuggingFace. Para la arquitectura central del sistema de detección deben implementar los componentes clave con PyTorch, no simplemente llamar una función de alto nivel que resuelva el problema entero.

¿Qué pasa si el dataset es demasiado grande para Google Colab? Usar un subconjunto justificado es completamente válido. Lo que no es válido es usar un subconjunto sin justificación o usar tan pocos datos que el modelo no aprenda nada significativo.

¿El MVP tiene que estar desplegado en internet? Sí. Un notebook local no cuenta como MVP. Si tienen problemas con Streamlit Cloud, un artifact de Claude con los pesos del modelo embebidos como JSON es una alternativa válida.

¿Pueden usar el mismo modelo para las dos etapas? Pueden compartir componentes entre etapas pero las dos etapas deben ser distinguibles: una que aprende sin etiquetas y otra que aprende con etiquetas. Un modelo que solo hace clasificación supervisada desde el inicio no cumple con el diseño del proyecto.